

Overview of Statistical Learning and Modeling – 36-600

Lab 10R: Pure Prediction: k -NN and SVM

Due Friday, November 7, 11:59pm

Trishla Nair

Data

Below we read in the breast-cancer dataset last seen in the PCA lab:

```
df      <- read.csv("https://raw.githubusercontent.com/FSKoerner/F25-36600-data/refs/heads/main/breast-cancer.csv")
response <- df[, 1]  # B for benign, M for malignant
predictors <- data.frame(scale(df[, -1]))
df      <- cbind(predictors, "Label" = response)
cat("Sample size: ", length(response), "\n")
```

```
## Sample size: 569
```

These data reside on [Kaggle](#). They provide information on breast cancer tumors (specifically, features extracted from the images of cells themselves!) for 569 people in which malignancy was suspected. The data are marked by *extreme* multicollinearity and redundancy: bad for inference, but fine for prediction! You'll code k -NN and SVM models for these data below.

Note that I scaled (i.e., standardized) the predictor data frame. As with many statistical learning methods, this is advised for both k -NN and SVM.

Also note: differentiating between the benign and malignant tumors turns out to be pretty easy using this data, so you will not see results that are substantially better, if at all better, than what you get when you learn a logistic regression model. The point today is the successful implementation, not to elicit the reaction “oh, wow, look how much better k -NN and SVM do!”

Question 1

Split the data and carry out a logistic regression analysis. (The response variable is dubbed `Label`.) Assume a class-separation threshold of 0.5, which is not optimal, but good enough, particularly since changing that threshold in the context of k -NN is difficult. (The optimal threshold would be nearer to 0.373. Why 0.373? The classes are imbalanced, and since B has more data - 62.7% of the data - and is labelled Class 0, the Class 1 probabilities will be systematically pulled downwards towards zero; a reasonable guess of threshold is $1 - 0.627 = 0.373$.)

```
set.seed(42)
s      <- sample(nrow(df), round(0.7 * nrow(df)))
df.train <- df[s, ]
df.test  <- df[-s, ]

logit.out <- glm(Label ~ ., data = df.train, family = binomial)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(logit.out)
```

```
##
## Call:
## glm(formula = Label ~ ., family = binomial, data = df.train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      2.7919      2.0389   1.369   0.171
## Mean.Radius     -49.6028     46.4419  -1.068   0.285
## Mean.Texture      1.4645      1.5185   0.964   0.335
## Mean.Perimeter    37.1284     45.7471   0.812   0.417
## Mean.Area         3.3962     20.2190   0.168   0.867
## Mean.Smoothness   1.6325      2.0081   0.813   0.416
## Mean.Compactness  -7.1230      5.4838  -1.299   0.194
## Mean.Concavity     1.6237      3.7513   0.433   0.665
## Mean.ConcavePoints 6.3644      5.3917   1.180   0.238
## Mean.Symmetry     -0.3572      1.2445  -0.287   0.774
## Mean.FractalDimension -1.4190      2.6410  -0.537   0.591
## Worst.Radius      12.7400     18.7022   0.681   0.496
## Worst.Texture      0.8068      1.4420   0.560   0.576
## Worst.Perimeter   -10.3636     12.3266  -0.841   0.400
## Worst.Area        19.3483     19.5849   0.988   0.323
## Worst.Smoothness   0.3287      1.3983   0.235   0.814
## Worst.Compactness  1.6693      3.1947   0.523   0.601
## Worst.Concavity    -0.5341      2.6295  -0.203   0.839
## Worst.ConcavePoints 1.5873      2.5064   0.633   0.527
## Worst.Symmetry     1.1967      1.2503   0.957   0.339
## Worst.FractalDimension 0.8508      2.3638   0.360   0.719
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 530.285  on 397  degrees of freedom
## Residual deviance:  36.438  on 377  degrees of freedom
## AIC: 78.438
##
## Number of Fisher Scoring iterations: 11
```

Question 2

Use the sample code in today's notes (specifically, that which was altered for classification - see slide 10) to implement a k -NN model. You'll want to plot the validation-set MCR versus k . A value of `k.max` of 30 should be sufficient.

Note 1: wherever it says `mse.k` in the notes, you'll instead want to use `mcr.k`, here in the classification setting (standing for "misclassification rate").

Note 2: the predictors are in columns 1-20 of `df.train` and `df.test`, and the response is in column 21.

```
library(FNN)
```

```
## Warning: package 'FNN' was built under R version 4.4.2
```

```

k.max <- 30
mcr.k <- rep(NA, k.max)
for (kk in 1:k.max) {
  knn.out <- knn(train = df.train[, 1:20], test = df.test[, 1:20], cl = df.train[, 21], k = kk)
  mcr.k[kk] <- mean(knn.out != df.test[, 21])
}

k.min <- which.min(mcr.k)
cat("The optimal number of nearest neighbors is", k.min)

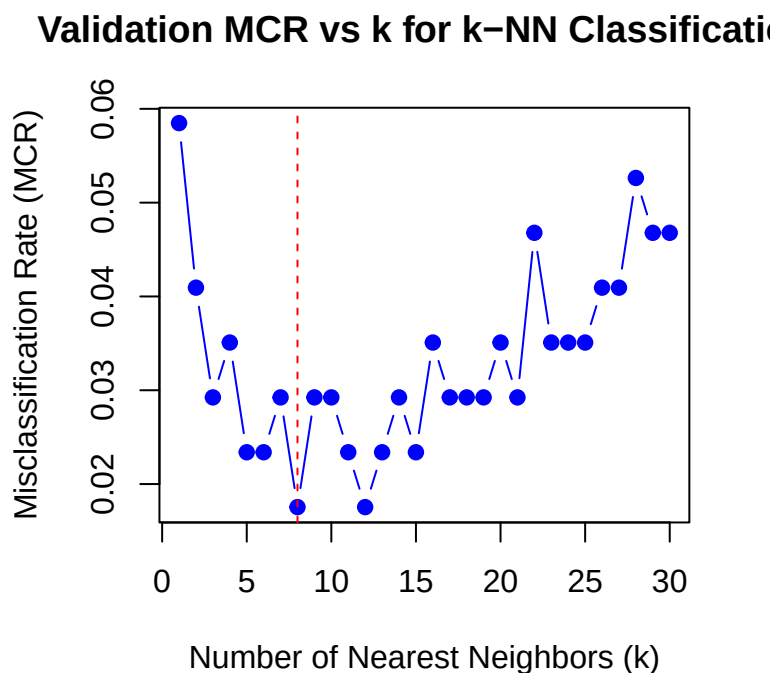
```

```
## The optimal number of nearest neighbors is 8
```

```

plot(1:k.max, mcr.k, type = "b", pch = 19, col = "blue",
     xlab = "Number of Nearest Neighbors (k)",
     ylab = "Misclassification Rate (MCR)",
     main = "Validation MCR vs k for k-NN Classification")
abline(v = k.min, col = "red", lty = 2)

```



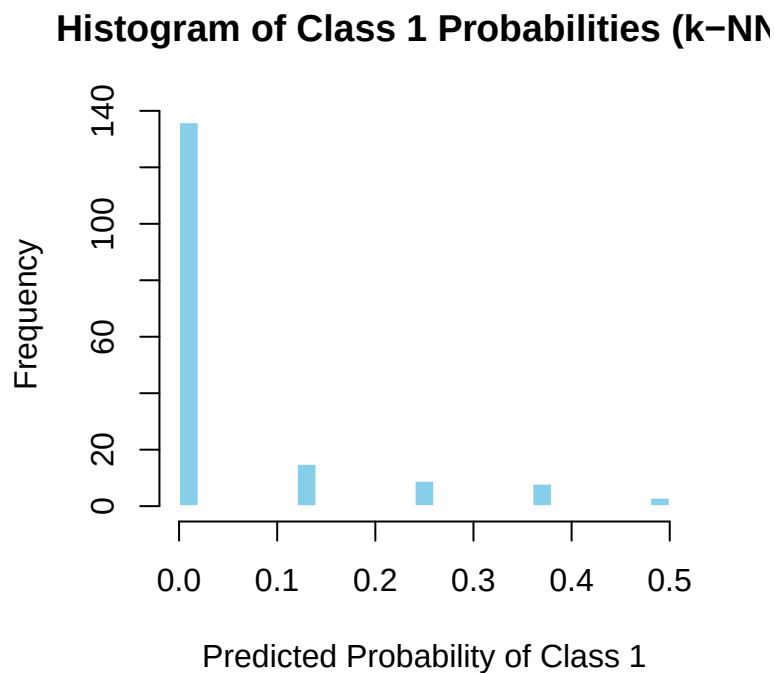
Question 3

Re-run the `knn()` function so as to be able to extract Class 1 probabilities. As with Question 2, here you can reference slide 10, but this time concentrate on adapting the code at the bottom. To demonstrate that you extracted the probabilities, simply make a histogram of them. You should observe two clear peaks: one at 0, and one at 1.

```
library(FNN)
k.opt <- k.min
knn.prob.out <- knn(train = df.train[, 1:20],
                    test  = df.test[, 1:20],
                    cl    = df.train[, 21],
                    k     = k.opt,
                    prob  = TRUE)
prob.knn <- ifelse(knn.prob.out == 1,
                  attr(knn.prob.out, "prob"),
                  1 - attr(knn.prob.out, "prob"))
summary(prob.knn)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.00000 0.00000 0.00000 0.05044 0.00000 0.50000
```

```
hist(prob.knn,
     breaks = 20,
     col = "skyblue",
     border = "white",
     main = "Histogram of Class 1 Probabilities (k-NN)",
     xlab = "Predicted Probability of Class 1")
```



Question 4

For access to SVM methods, we will work with the `e1071` package. (Its name comes from the course coding used by the Institute of Statistics and Probability Theory at the Technische Universität Wien, in Vienna.)

It's like us calling a package 36-600. Which perhaps we should.) Here, code a support vector classifier (meaning, do SVM with `kernel = "linear"`): use the `tune()` function with a representative sequence of potential costs C , then extract the best model. If the optimum value of C occurs at or very near the end of your sequence of potential costs, alter the sequence to extend further. The variable `best.parameters`, embedded in the output, provides the optimal value for C . Report that value. Use the best model to generate predictions, a test-set MCR, and a confusion matrix.

Note that `tune()` does cross-validation on the training set to estimate the optimum value of C . Which means that the training data are randomly assigned to folds (by default, 10 folds - to change this, you'd provide tuning parameters to the function that sets them `tune.control()`, e.g., via `tune.control(cross = 5)`). This random sampling inherent to the algorithm means you should set a random number seed before calling `tune()` for reproducibility.

See the last code block of page 390 of ISLR (2nd edition) for an example of how to specify ranges of tuning parameters. Note that here, there is only one: `cost`. As for prediction: `tune()` will return an object that includes `best.model`. Pass this model to `predict()` along with the argument `newdata = .` By default, `predict()` will output a vector of class predictions, so there is no need for rounding in determining classes.

```
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.4.3
```

```
set.seed(42)
cost.values <- 10^seq(-3, 3, by = 1)
tune.out.lin <- tune(svm, Label ~ ., data = df.train, kernel = "linear", ranges = list(cost = cost.values))
summary(tune.out.lin)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##   cost
##   0.1
##
## - best performance: 0.01762821
##
## - Detailed performance results:
##   cost      error dispersion
## 1 1e-03 0.07032051 0.03068163
## 2 1e-02 0.03519231 0.02939638
## 3 1e-01 0.01762821 0.02080949
## 4 1e+00 0.02512821 0.02899345
## 5 1e+01 0.03762821 0.02952594
## 6 1e+02 0.04012821 0.03379204
## 7 1e+03 0.04512821 0.03691477
```

```
best.lin.model <- tune.out.lin$best.model
best.lin.C <- tune.out.lin$best.parameters$cost
cat("Optimal cost (C):", best.lin.C, "\n")
```

```
## Optimal cost (C): 0.1
```

```
pred.lin <- predict(best.lin.model, newdata = df.test)
mcr.lin <- mean(pred.lin != df.test$Label)
cat("Test-set MCR:", round(mcr.lin, 4), "\n")
```

```
## Test-set MCR: 0.0292
```

```
table(Predicted = pred.lin, Actual = df.test$Label)
```

```
##           Actual
## Predicted  B    M
##           B 110   3
##           M   2  56
```

Question 5

Now code a support vector machine with a polynomial kernel. In addition to tuning `cost`, you also have to tune the polynomial `degree`. Try integers from 2 up to some maximum number (not too large, like 4). *Note:* if you get the warning `WARNING: reaching max number of iterations`, don't worry about it - the function will still return the best estimates it can.

```
set.seed(42)
tune.out.poly <- tune(svm, Label ~ ., data = df.train, kernel = "polynomial", ranges = list(cost = 10^seq(-2, 2), degree = 2:4))
summary(tune.out.poly)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##   cost degree
##   100      3
##
## - best performance: 0.05019231
##
## - Detailed performance results:
##   cost degree      error dispersion
## 1  1e-02      2 0.34692308 0.03889687
## 2  1e-01      2 0.27628205 0.04525780
## 3  1e+00      2 0.22108974 0.05233468
## 4  1e+01      2 0.17833333 0.07099023
## 5  1e+02      2 0.16070513 0.08077322
## 6  1e+03      2 0.17320513 0.07272859
## 7  1e-02      3 0.26615385 0.03239530
## 8  1e-01      3 0.15814103 0.04366828
## 9  1e+00      3 0.08788462 0.04445621
## 10 1e+01      3 0.05025641 0.02635786
## 11 1e+02      3 0.05019231 0.03328268
## 12 1e+03      3 0.06275641 0.04290293
## 13 1e-02      4 0.30916667 0.03691829
```

```
## 14 1e-01      4 0.25615385 0.04140031
## 15 1e+00      4 0.21602564 0.03115264
## 16 1e+01      4 0.19339744 0.06222320
## 17 1e+02      4 0.18820513 0.07611650
## 18 1e+03      4 0.23076923 0.10918532
```

```
best.poly.model <- tune.out.poly$best.model
best.poly.params <- tune.out.poly$best.parameters
cat("Optimal cost (C):", best.poly.params$cost,
    " | Optimal degree:", best.poly.params$degree)
```

```
## Optimal cost (C): 100 | Optimal degree: 3
```

```
pred.poly <- predict(best.poly.model, newdata = df.test)
mcr.poly <- mean(pred.poly != df.test$Label)
cat("Test-set MCR:", round(mcr.poly, 4), "\n")
```

```
## Test-set MCR: 0.0409
```

```
table(Predicted = pred.poly, Actual = df.test$Label)
```

```
##           Actual
## Predicted  B    M
##           B 112   7
##           M   0  52
```

Question 6

Now code a support vector machine with a radial kernel. In addition to tuning `cost`, you also have to tune the parameter `gamma`. Try a base-10 logarithmic sequence of values that encompasses -8 (for 10^{-8}).

```
set.seed(42)
tune.out.rad <- tune(svm,
  Label ~ .,
  data = df.train,
  kernel = "radial",
  ranges = list(cost = 10^seq(-2, 3, by = 1),
    gamma = 10^seq(-8, 0, by = 2)))

summary(tune.out.rad)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##   cost gamma
## 1000 1e-04
##
```

```
## - best performance: 0.02012821
##
## - Detailed performance results:
##      cost gamma      error dispersion
## 1  1e-02 1e-08 0.38461538 0.04890689
## 2  1e-01 1e-08 0.38461538 0.04890689
## 3  1e+00 1e-08 0.38461538 0.04890689
## 4  1e+01 1e-08 0.38461538 0.04890689
## 5  1e+02 1e-08 0.38461538 0.04890689
## 6  1e+03 1e-08 0.38461538 0.04890689
## 7  1e-02 1e-06 0.38461538 0.04890689
## 8  1e-01 1e-06 0.38461538 0.04890689
## 9  1e+00 1e-06 0.38461538 0.04890689
## 10 1e+01 1e-06 0.38461538 0.04890689
## 11 1e+02 1e-06 0.30429487 0.03529466
## 12 1e+03 1e-06 0.05532051 0.02851592
## 13 1e-02 1e-04 0.38461538 0.04890689
## 14 1e-01 1e-04 0.38461538 0.04890689
## 15 1e+00 1e-04 0.30679487 0.03583786
## 16 1e+01 1e-04 0.05532051 0.02851592
## 17 1e+02 1e-04 0.02269231 0.02502866
## 18 1e+03 1e-04 0.02012821 0.01993992
## 19 1e-02 1e-02 0.38461538 0.04890689
## 20 1e-01 1e-02 0.06282051 0.03587453
## 21 1e+00 1e-02 0.02519231 0.02372478
## 22 1e+01 1e-02 0.02012821 0.02316224
## 23 1e+02 1e-02 0.03262821 0.03125865
## 24 1e+03 1e-02 0.04775641 0.02489523
## 25 1e-02 1e+00 0.38461538 0.04890689
## 26 1e-01 1e+00 0.38461538 0.04890689
## 27 1e+00 1e+00 0.27160256 0.07167143
## 28 1e+01 1e+00 0.25653846 0.07358237
## 29 1e+02 1e+00 0.25653846 0.07358237
## 30 1e+03 1e+00 0.25653846 0.07358237
```

```
best.rad.model <- tune.out.rad$best.model
best.rad.params <- tune.out.rad$best.parameters
cat("Optimal cost (C):", best.rad.params$cost,
    " | Optimal gamma:", best.rad.params$gamma)
```

```
## Optimal cost (C): 1000 | Optimal gamma: 1e-04
```

```
pred.rad <- predict(best.rad.model, newdata = df.test)
mcr.rad <- mean(pred.rad != df.test$Label)
cat("Test-set MCR:", round(mcr.rad, 4))
```

```
## Test-set MCR: 0.0292
```

```
table(Predicted = pred.rad, Actual = df.test$Label)
```

```
##      Actual
## Predicted  B  M
##          B 110  3
##          M   2 56
```

Question 7

Re-run the `tune()` and `predict()` functions so as to be able to extract Class 1 probabilities. You may want to refer back to the final bullet point on Slide 17. To demonstrate that you extracted the probabilities, simply make a histogram of them. You should observe two clear peaks...one at 0, and one at 1.

```
set.seed(42)
svm.prob.model <- svm(Label ~ .,
  data = df.train,
  kernel = "radial",
  cost = best.rad.params$cost,
  gamma = best.rad.params$gamma,
  probability = TRUE)
pred.prob <- predict(svm.prob.model, newdata = df.test, probability = TRUE)
prob.mat <- attr(pred.prob, "probabilities")

prob.class1 <- prob.mat[, "M"]
hist(prob.class1,
  breaks = 20,
  col = "skyblue",
  border = "white",
  main = "Histogram of Class 1 Probabilities (SVM)",
  xlab = "Predicted Probability of Class 1")
```

Histogram of Class 1 Probabilities (SVM)

